CS336 作业 5：推理强化学习

版本 26.0.0 · CS336 课程组 · 2026 年春季

译文说明：完整翻译作业说明；题目标识、分值、公式编号、接口和实验参数均按原文保留。代码、公式及英文实验提示词保留原样，提示词另附中文释义。本文件不包含习题解答。

1 作业概述

本次作业将带你训练语言模型，使其能够推理并解决下游任务。

你将实现的内容

1. 零样本、少样本与思维链提示。

2. 组相对策略优化（Group Relative Policy Optimization，GRPO）：利用外部奖励改善模型性能的强化学习算法。

3. 策略梯度估计的不同变体，探索方差降低与重要性权重裁剪策略。

你将运行的实验

1. 测量 OLMo-2-0425-1B 在 GSM8K 上使用提示词的表现。

2. 使用同策略（on-policy）GRPO 改善该模型在 GSM8K 上的表现。

3. 运行 RFT、Dr. GRPO、MaxRL 等变体，探索 RL 算法选择。

4. 运行异策略（off-policy）GRPO，加速训练并探索不同裁剪策略。

代码结构

代码与说明位于 https://github.com/stanford-cs336/assignment5-alignment 。请用 git clone 克隆；收到更新通知后用 git pull 获取最新版。

1. cs336_alignment/*：本作业实现目录，除下述起始代码外基本为空，可以从零开始。

2. cs336_alignment/vllm_utils.py：启动 vLLM 服务、生成文本和同步权重的代码。

3. cs336_alignment/drgrpo_grader.py：评判数学问题模型输出的代码。

4. cs336_alignment/prompts/*：提供的提示词文本文件。

5. tests/*.py：包含必做 GRPO 测试与可选对齐 / 安全补充材料的测试。

必做 tests/test_grpo.py 调用 tests/adapters.py 中的钩子，你需要实现适配器接入自己的代码。可新增或修改测试辅助调试，但最终必须通过原始测试套件。

README.md 包含基本环境配置说明。

如何提交

向 Gradescope 提交 writeup.pdf，排版回答所有书面问题；另提交 code.zip，包含你编写的全部代码。运行 test_and_make_submission.sh 生成 code.zip。

2 引言

2.1 背景

前四次作业学习了如何预训练基础模型。现在学习后训练（post-training）：有了基础模型，怎样把它变成能解决下游任务的有用工具？

后训练的一部分是对齐。由于预训练的数据混合与目标，基础模型会获得广泛的知识和行为。但解决任务时，我们希望它表现为有帮助、无害的助手。把广泛的基础模型转变成专注的聊天模型，称为“指令微调”或“对齐”，将在作业 5 的可选补充中介绍。

另一部分是强化学习。预训练旨在获得广泛知识，所以使用网页等广泛数据；后训练 RL 的目标更具体，例如提高数学解题准确率。这与预训练至少有两点不同：(a) 可用数据不再那么多；(b) 目标从“覆盖广泛知识”转向“生成准确回答”。因此需要强化学习这一新方法。

RL 提供一个问题数据集和用于判断回答是否正确的评分函数。例如，编程任务可要求“写一个反转列表的 Python 函数”，评分函数是一组测试，如 assert f([0, 1, 2]) == [2, 1, 0]；数学任务可问“四年前 Tom 的年龄是 John 的一半，John 现在 20 岁，Tom 现在几岁？”，评分函数解析最终答案并检查是否为 12。

与预训练不同，这里没有供模仿的回答数据集：编程例中没有标准程序，数学例中没有产生答案的正确推理链。RL 直接以模型准确率为目标做梯度更新。总体流程是从模型采样回答、评分，再提高正确回答的权重。

本作业同时从数学和实验角度学习 RL。RL 既慢又不稳定，研究也很难：不同随机种子的结果方差大，看似微小的实现细节影响显著。我们将介绍大语言模型 RL，并探索这些挑战。

2.2 模型与数据集

使用 OLMo-2-0425-1B，模型地址：https://huggingface.co/allenai/OLMo-2-0425-1B 。它先在 OLMo-mix-1124 上预训练，再在 Dolmino-mix-1124 上进行中期训练，总计 4 万亿 token。

OLMo-mix-1124 主要由课堂介绍过的 DCLM-Baseline [J. Li 等，2024] 构成；Dolmino-mix-1124 更聚焦，约 50% 为 DCLM，另 50% 为指令跟随、数学、代码、STEM 论文与 wiki 数据。数据集分别见 https://huggingface.co/datasets/allenai/olmo-mix-1124 和 https://huggingface.co/datasets/allenai/dolmino-mix-1124 。更多细节见 OLMo 2 技术报告 [T. OLMo 等，2024]。学到这里，你应已有理解各项设计决策的背景知识。

下游使用 GSM8K [K. Cobbe 等，2021]，位于 data/gsm8k/train.jsonl 与 data/gsm8k/test.jsonl，也可在线获取：https://huggingface.co/datasets/openai/gsm8k 。它是一组相对简单的小学数学推理应用题，例如：

```
{
  "question": "Natalia sold clips to 48 of her friends in April, and then she sold half as
many clips in May. How many clips did Natalia sell altogether in April and May?",
  "answer": "Natalia sold 48/2 = <<48/2=24>>24 clips in May.\nNatalia sold 48+24 =
<<48+24=72>>72 clips altogether in April and May.\n#### 72"
}
```

原文第 3 页

示例中文释义：Natalia 四月卖出 48 个发夹，五月卖出四月的一半，两个月共卖出多少？给定答案先算五月为 48/2=24，再算总数 48+24=72，以 #### 72 结尾。原始 JSON 及分隔符按上方保留。

RL 将让模型学会为这类问题生成推理链，提高数学解题能力。

由于资源限制，我们只选一个小规模模型与数据集组合作为实验平台。该小模型在非常大量的 token 上训练过，token 数达到参数量的 4000 倍，因此能力较强，可以在真实数据集上以小规模观察到合理的 RL 效果。课程中自行训练的模型遗憾地还不够强，难以解数学题。完成作业后，也可尝试把自己的模型用于更简单任务的 RL。

RL 的训练动态高度依赖模型和数据集，因此本作业的观察未必能迁移到其他组合。

2.3 符号约定

语言建模与 RL 常对同一对象使用不同术语。以下列出后文符号及对应含义，可随时回查。

• ρ：LM 中的提示词分布 / 数据集；RL 中的初始状态分布。表示问题的分布。

• x：提示词 / 问题；RL 中为初始状态。从 ρ 采样得到。

• y：回答、补全、生成结果或样本；RL 中为 rollout、轨迹或采样动作序列。是针对 x 采样的答案。

• y_t：第 t 个生成 token；RL 中的动作。

• y_<t：前缀，即位置 t 之前生成的所有 token，y_1,…,y_(t-1)。

• π_θ：参数为 θ 的模型；RL 中的策略。给定 x，为回答 y 赋予概率 π_θ(y|x)。

• π_θ(y_t|x,y_<t)：下一 token 分布；RL 中的 t 时刻策略。给定问题和此前生成 token 后，y_t 的条件概率。

• r(y|x)：奖励；回答正确性的标量评分，本作业为 0 或 1。

• B：每个推理批次中的提示词数量。

• G：每个提示词采样的回答数；RL 中称组大小。

• len(y) 或 L：回答长度；RL 中的时域长度（horizon），即生成 token 数。

• A^(i,j)：优势（advantage）；减去基线并归一化后，赋予第 i 个问题的第 j 个回答的权重。

3 提示词

将基础模型用于下游任务的第一步是构造提示词。基础模型学过广泛行为，提示是一种轻量方式，使其转向解题模式。后面还会看到，提示词也塑造 RL 动态和探索过程。

最简单方式是提供问题，并从下一 token 分布采样回答，称为 question_only。将它与 r1_zero 比较：后者除问题外，还指示模型做思维链推理 [DeepSeek-AI 等，2025]。

3.1 使用 vLLM 推理

生成回答需要推理引擎。实现引擎超出本作业范围，因此使用 vLLM [W. Kwon 等，2023]，其优化包括快速 CUDA 内核、用于高效 KV 缓存的 PagedAttention 等。cs336_alignment/vllm_utils.py 已提供启动服务和生成的代码，接口如下：

```python
@dataclass
class VLLMCompletion:
    text: str
    token_ids: list[int]
    finish_reason: str | None


@dataclass
class VLLMServer:
    model_id: str
    gpu: int = 0
    seed: int = 0
    gpu_memory_utilization: float = 0.9
```

原文第 4 页

```python
    def start(self) -> None: ...

    def generate_completions(
        self,
        prompts: list[str],
        sampling_params: dict,
        batch_size: int | None = None,
    ) -> list[VLLMCompletion]: ...
```

原文第 5 页

3.2 零样本、少样本与思维链提示

除非另有说明，GSM8K 实验使用 DeepSeek R1-Zero 的以下提示，称为 r1_zero：

```
A conversation between User and Assistant. The User asks a question, and the Assistant solves
it. The Assistant first thinks about the reasoning process in the mind and then provides the
User with the answer. The reasoning process is enclosed within <think> </think> and the
answer is enclosed within <answer> </answer> tags, respectively, i.e., <think> reasoning
process here </think> <answer> answer here </answer>.
User: {question}
Assistant: <think>
```

原文第 5 页

中文释义：用户与助手对话，用户提出问题，助手解答。助手先在内部思考推理过程，再给答案；推理过程置于 <think>...</think>，答案置于 <answer>...</answer>。最后给出 User: {question}，并以 Assistant: <think> 引导模型开始推理。

提示文件位于 cs336_alignment/prompts/r1_zero.prompt。question 是插入的问题，例如前面的发夹题。期待模型扮演助手，从已经给出的 <think> 开始生成推理，以 </think> 结束，再在 answer 标签中给出最终符号答案，例如 <answer> 4x + 10 </answer>。

标签便于解析输出并与标准答案比较，也方便在生成 </answer> 时停止。

少样本提示则是在实际问题前添加几个问答示例。r1_zero 的少样本版本如下：

```
A conversation between User and Assistant. The User asks a question, and the Assistant solves
it. The Assistant first thinks about the reasoning process in the mind and then provides the
User with the answer. The reasoning process is enclosed within <think> </think> and the
answer is enclosed within <answer> </answer> tags, respectively, i.e., <think> reasoning
process here </think> <answer> answer here </answer>.
User: {question-1}
Assistant: <think> {reasoning-1} </think> <answer> {answer-1} </answer>
User: {question-2}
Assistant: <think> {reasoning-2} </think> <answer> {answer-2} </answer>
User: {question-3}
Assistant: <think> {reasoning-3} </think> <answer> {answer-3} </answer>
User: {question}
Assistant: <think>
```

原文第 5 页

中文结构：先使用同样的对话说明，再依次给出 question-1 / reasoning-1 / answer-1、question-2 / reasoning-2 / answer-2、question-3 / reasoning-3 / answer-3，最后放入待解问题，并以 Assistant: <think> 结尾。

少样本通过展示任务示例改善性能。公开基础模型报告的基准成绩常采用少样本；例如 OLMo-2-0425-1B 模型卡上的 GSM8K 结果使用 8-shot。我们在 cs336_alignment/prompts/r1_zero_three_shot_gsm8k.prompt 中提供 3-shot 版本，示例来自 OLMES 仓库：https://github.com/allenai/olmes/blob/main/oe_eval/tasks/fewshot_sources.py#L2247 。

作为基线，还使用 cs336_alignment/prompts/question_only.prompt 中的 question_only：

```
{question} Please put your final answer within \\boxed{{}}.
```

原文第 6 页

中文释义：给出 {question}，并要求“请将最终答案放在 boxed 标记中”。虽然名为 question_only，仍要求答案加框，以便评分器解析。实验应使用上方原始英文模板与转义形式。

3.3 评分函数

模型生成后，需要检查是否正确。数学题有标准答案，例如 0.5，但模型可能以 <answer> 1/2 </answer> 或“The answer is 0.5”等方式正确回答。因此需要解析函数，接收模型输出与已知答案，返回正确与否。

本实验使用近期推理 RL 工作 [Z. Liu 等，2025] 中快速且较准确的解析器。r1_zero 对应 cs336_alignment.drgrpo_grader.r1_zero_reward_fn。question_only 不要求 think 和 answer 标签，应使用同文件中的 cs336_alignment.drgrpo_grader.question_only_reward_fn，后者查找 \boxed{} 中的最终答案。

奖励函数返回总奖励，以及独立的格式奖励与答案奖励，分别表示格式是否正确和解析出的答案是否正确。有些工作对格式正确但答案错误的回答给部分分数；本实验不这样做，总奖励直接等于答案奖励，格式奖励只用于日志。

ground_truth 参数应只包含答案。GSM8K 的原始答案形式为 {rationale} #### {answer}，应按 #### 分割并去除首尾空白。

3.4 实验

现在评估基础模型的提示性能。

生成超参数：temperature=1.0，top-p=1.0，最大生成长度 512。r1_zero 要求以 </answer> 结束，可设置 vLLM 遇到该字符串停止：

```
# Based on Dr. GRPO: stop when the model completes its answer
# https://github.com/sail-sg/understand-r1-zero/blob/
#   c18804602b85da9e88b4aeeb6c43e2f08c594fbc/train_zero_math.py#L167
sampling_params['stop'] = ["</answer>"]
sampling_params['include_stop_str_in_output'] = True
```

原文第 6 页

这一停止字符串仅用于 r1_zero 和 r1_zero_three_shot，不用于 question_only。

题目（prompting_baselines）：在 GSM8K 上运行 OLMo-2-0425-1B（5 分）

(a) 编写脚本，分别用零样本 question_only、零样本 r1_zero 和少样本 r1_zero_three_shot 评估模型。

运行并观察输出。对每个提示词，统计：(1) 格式与正确性奖励都为 1；(2) 格式奖励为 1、正确性为 0；(3) 两者均为 0。至少观察第 2 类的 10 个例子，其中多少实际正确但解析失败？第 3 类呢？

提交内容：几句话的分析、评估指标，以及若干提示词与回答示例。

(b) 根据输出描述各提示词下的行为。若希望模型回答问题，仅提供问题是否足够，还是会产生其他行为？零样本 r1_zero 和少样本版本如何塑造模型行为？

提交内容：几句话的评论及支持示例。

4 组相对策略优化

测量仅靠提示的表现后，用训练进一步改善。希望优化准确率，等价于期望奖励：

$$J_\theta = E_{x\sim\rho} E_{y\sim\pi_\theta(y\mid x)}[r(y\mid x)], \tag{1}$$

原文第 7 页

ρ 是问题 x 的任务分布，π_θ 是模型，y 是为 x 采样的回答，r(y|x) 表示答案是否正确，也称奖励函数。

该目标与预训练明显不同：交叉熵 E_(x~D)[-log π_θ(x)] 从数据集采样，而准确率目标对模型自身生成的样本取期望，因此需要强化学习等新工具。

总体上，RL 从模型采样、评分，并强化正确结果。RL 通常慢且困难，作业将探索让它更快、更稳定的方法。

由于 RL 涉及从模型采样，提示词会影响动态。一个令人兴奋的发现是：对较强基础模型使用思维链提示并进行 RL，能够显著提高推理能力 [OpenAI 等，2024；DeepSeek-AI 等，2025]。因此除核心算法，也会探索 RL 与思维链提示的相互作用。

4.1 推导同策略 GRPO

先研究 RL 数学，逐步建立语言模型中常用的 GRPO [Z. Shao 等，2024]。本节主要参考 OpenAI 的 Spinning Up in Deep RL [J. Achiam，2018] 与 Nathan Lambert 的 RLHF Book [N. Lambert，2024]，两者有更深入讲解。

4.1.1 将语言模型视为策略

传统 RL 中，策略接收状态 s_t，输出动作 a_t，产生即时奖励 r_t=r(s_t,a_t)，并从 next_state(s_t,a_t) 得到下一状态 s_(t+1)。RL 优化策略以获得高奖励。

参数为 θ 的因果语言模型 π_θ，根据前缀 y_<t=(y_1,…,y_(t-1)) 给下一 token y_t 定义概率分布。可将下一 token 当动作、当前前缀当状态，因此 LM 是离散类别上的随机策略：

$$a_t \sim \pi_\theta(\cdot \mid s_t), \pi_\theta(a_t \mid s_t) = [\text{softmax}\ (f_\theta(s_t))]_{a_t}. \tag{2}$$

原文第 8 页

策略梯度优化需要两个原语：从上述类别分布采样动作 a_t；计算动作对数似然 log π_θ(a_t|s_t)。

LLM RL 中，s_t 是迄今生成的部分答案，a_t 是下一个 token；生成 <|end_of_text|> 等结束标记时，episode 结束。r1_zero 使用 </answer> 作为结束标记。

4.1.2 轨迹

从 s_0~ρ 开始，根据策略采样动作，再按状态转移分布得到下一状态，重复这一过程，形成有限时域轨迹：

$$\tau = (s_0, a_0, s_1, a_1, ..., s_T, a_T), \tag{3}$$

原文第 8 页

T 表示轨迹长度，a_T 是结束 token，或此时已达到最大生成 token 预算。

本作业 s_0 是含数学题的提示 x，可能包含 CoT 或少样本内容。之后逐 token 采样，下一状态就是旧前缀拼接新 token，即 s_(t+1)=(s_t,a_t)。轨迹也称 episode 或 rollout，后文交替使用。

4.1.3 奖励与回报

标量奖励 r_t=r(s_t,a_t) 衡量状态 s_t 下所采取动作的即时质量。数学应用题等可验证任务中，中间推理步骤奖励为零，直到输出最终答案，终止动作才得到验证奖励：

$$r_T = r(s_T, a_T) := \begin{cases} 1 \text{ if trajectory } \ \tau \ \text{ matches the ground-truth according to our reward function} \\ 0 \text{ otherwise} \end{cases}. \tag{4}$$

原文第 8 页

式 (4) 中文含义：轨迹 τ 按奖励函数判断与标准答案一致则奖励为 1，否则为 0。本作业最终答案正确则 r_T=1，否则为 0。回报 R(τ) 汇总沿轨迹的奖励，这里等于 r_T。

智能体目标是最大化期望回报：

$$J_\theta = E_{\tau \sim \pi_\theta}[r(\tau)], \tag{5}$$

原文第 9 页

τ~π_θ 表示先从 ρ 采样初始状态，再按策略采样动作、按转移分布采样状态所形成的轨迹分布。本作业就是从数据集采样数学问题 x，再逐 token 生成直到回答结束，记为 y~π_θ(y|x)。优化问题为：

$$\theta^* = \arg\max_\theta J_\theta. \tag{6}$$

原文第 9 页

4.1.4 策略梯度

下面学习策略梯度算法。后文统一用 LM 记号 (x,y) 表示提示与回答，不再主要使用状态 s_t、动作 a_t。

希望优化模型期望奖励，即准确率：

$$J_\theta = E_{x\sim\rho} E_{y\sim\pi_\theta(y\mid x)}[r(y\mid x)], \tag{7}$$

原文第 9 页

r(y|x) 表示 y 是否正确。使用梯度上升：

$$\theta_{k+1} = \theta_k + \alpha \nabla_\theta J_{\theta_k}, \tag{8}$$

原文第 9 页

因此需要从样本估计 ∇_θ J。将梯度改写为对样本的期望，会得到以 REINFORCE 论文 [R. J. Williams，1992] 命名的策略梯度：

$$\nabla_\theta E_{x\sim\rho} E_{y\sim\pi_\theta(y\mid x)}[r(y\mid x)] = E_{x\sim\rho} E_{y\sim\pi_\theta(y\mid x)}[r(y\mid x)\nabla_\theta \log \pi_\theta(y\mid x)]. \tag{9}$$

原文第 9 页

这给出直观算法：从数据集采样问题，从模型采样回答，计算 r(y|x)∇_θ log π_θ(y|x) 的样本均值，再进行梯度更新。该表达式提高高奖励回答的对数概率。反复采样和更新，就形成基本 RL 训练循环。

推导利用对数导数技巧：log f(x) 的导数为 f′(x)/f(x)，因此：

$$\nabla_\theta \pi_\theta(y\mid x) = \pi_\theta(y\mid x)\nabla_\theta \log \pi_\theta(y\mid x). \tag{10}$$

原文第 9 页

直接应用恒等式得到：

$$\begin{aligned}
\nabla_\theta E_{y\sim\pi_\theta(y\mid x)}[r(y\mid x)] &= \sum_y \nabla_\theta \pi_\theta(y\mid x) r(y\mid x) && (11)\\
&= \sum_y \pi_\theta(y\mid x)\nabla_\theta \log \pi_\theta(y\mid x) r(y\mid x) && (12)\\
&= E_{y\sim\pi_\theta(y\mid x)}[r(y\mid x)\nabla_\theta \log \pi_\theta(y\mid x)]. && (13)
\end{aligned}$$

原文第 9 页

这里省略了 E_(x~ρ)，因为它与 ∇_θ 可交换。

给定 B 个独立同分布采样的提示 x^(1),…,x^(B)，每个提示再独立采样 G 个回答 y^(i,j)，策略梯度样本估计为：

$$\hat{g} \leftarrow \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G} r\big(y^{(i,j)}\mid x^{(i)}\big)\nabla_\theta \log \pi_\theta\big(y^{(i,j)}\mid x^{(i)}\big), \tag{14}$$

原文第 10 页

它提高高奖励回答的权重。下面从该估计出发，逐项修改，得到 GRPO。

4.1.5 基线

基本 REINFORCE 估计的期望正确，但方差大，导致 RL 不稳定。基线用于稳定训练和降低方差。最简单的基线是从奖励减去常数 b：

$$E_{x\sim\rho}E_{y\sim\pi_\theta(y\mid x)}[(r(y\mid x)-b)\nabla_\theta\log\pi_\theta(y\mid x)]. \tag{15}$$

原文第 10 页

例如二元奖励下取 b=0.5，正确回答权重为 0.5，错误回答权重为 -0.5；原始估计则对正确回答赋权 1，对错误回答赋权 0、不作更新。

关键结论：只要 b 不依赖动作 / 回答 y，减去 b 就不改变估计期望。b 可以是常数、x 的函数或策略的函数。有：

$$E_{x\sim\rho}E_{y\sim\pi_\theta(y\mid x)}[(r(y\mid x)-b)\nabla_\theta\log\pi_\theta(y\mid x)]=E_{x\sim\rho}E_{y\sim\pi_\theta(y\mid x)}[r(y\mid x)\nabla_\theta\log\pi_\theta(y\mid x)]. \tag{16}$$

原文第 10 页

该结论直接来自：

$$E_{y\sim\pi_\theta(y\mid x)}[\nabla_\theta\log\pi_\theta(y\mid x)]=\nabla_\theta E_{y\sim\pi_\theta(y\mid x)}[1]=0, \tag{17}$$

原文第 10 页

第一步逆向使用前面的对数导数技巧。

基线保持期望，却可能增大或减小方差。将估计写成 (1/n)Σ_i Z_i，单个 Z_i 为样本 (x,y) 的 (r(y|x)-b)∇_θ log π_θ(y|x)，则估计方差为 (E[Z_i²]-E[Z_i]²)/n。下面探索基线何时增减方差。

题目（baseline_calcs）：计算策略梯度估计的方差（5 分）

π_θ 定义二元动作空间 A={0,1} 上的策略，π_θ(A=1)=p=σ(θ)，σ(θ)=1/(1+e^(-θ))。奖励 r(A)=1{A=1}，动作 1 得 1，否则为 0。

(a) 对 n 个独立样本 A_i~π_θ，策略梯度估计如下。方差是多少？

$$\frac{1}{n}\sum_{i=1}^{n}r(A_i)\nabla_\theta\log\pi_\theta(A_i) \tag{18}$$

原文第 10 页

提交内容：用 n、p 表示的表达式及推导。

(b) 对同样 n 个独立样本，基线调整后的估计如下。方差是多少？

$$\frac{1}{n}\sum_{i=1}^{n}(r(A_i)-b)\nabla_\theta\log\pi_\theta(A_i) \tag{19}$$

原文第 11 页

提交内容：用 n、b、p 表示的表达式、推导与讨论。

(c) 代入“总体均值”基线 b=p，方差是多少？与未调整估计相比，是始终更小、始终更大，还是随 p 而变化？

GRPO 中的基线

GRPO 首先使用“组均值”基线：

$$\hat{g} \leftarrow \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\left(r\left(y^{(i,j)} \mid x^{(i)}\right)-\mu_i\right)\nabla_\theta \log \pi_\theta\left(y^{(i,j)} \mid x^{(i)}\right) \tag{20}$$

原文第 11 页

μ_i=(1/G)Σ_j r(y^(i,j)|x^(i))，即模型对同一提示的平均奖励。虽然 μ_i 依赖回答 y，期望仍只差一个 (G-1)/G 的缩放：

$$E_{y^{(1)},\ldots,y^{(G)}\sim\pi_\theta(y\mid x)}\left[\frac{1}{G}\sum_{j=1}^{G}\left(r\left(y^{(j)} \mid x\right)-\mu_i\right)\nabla_\theta \log \pi_\theta\left(y^{(j)} \mid x\right)\right] \tag{21}$$

$$=\frac{1}{G}\sum_{j=1}^{G}E_{y^{(2)},\ldots,y^{(G)}\sim\pi_\theta(y\mid x)}\left[E_{y^{(1)}\sim\pi_\theta(y\mid x)}\left[\left(r\left(y^{(1)} \mid x\right)-\mu_i\right)\nabla_\theta \log \pi_\theta\left(y^{(1)} \mid x\right)\right]\right] \tag{22}$$

$$=\frac{1}{G}\sum_{j=1}^{G}E_{y^{(1)}\sim\pi_\theta(y\mid x)}\left[\left(r\left(y^{(1)} \mid x\right)-\frac{1}{G}r\left(y^{(1)} \mid x\right)\right)\nabla_\theta \log \pi_\theta\left(y^{(1)} \mid x\right)\right] \tag{23}$$

$$=\frac{G-1}{G}E_{y\sim\pi_\theta(y\mid x)}[r(y \mid x)\nabla_\theta \log \pi_\theta(y \mid x)]. \tag{24}$$

原文第 11 页

经组均值调整的奖励常称为优势，表示这条 rollout 相对于平均奖励的优势，而非绝对奖励。

4.1.6 优势归一化

减去组均值后，GRPO 再除以组标准差：

$$\hat{g} \leftarrow \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{r\left(y^{(i,j)} \mid x^{(i)}\right)-\mu_i}{\mathrm{std}_i}\nabla_\theta \log \pi_\theta\left(y^{(i,j)} \mid x^{(i)}\right), \tag{25}$$

原文第 11 页

std_i=√[(1/G)Σ_j(r(y^(i,j)|x^(i))-μ_i)²]。torch.std 默认采用带偏差修正、略有不同的样本标准差；实现时应使用默认 torch.std。

除以标准差不再保持估计期望，因此已不再对原始期望奖励 J_θ 做梯度上升。可将它视为稳定性技巧：若假设各梯度向量独立同分布且为高斯，按组标准差缩放能使各组梯度更新的范数大致相当。后面进一步探讨。

4.1.7 序列归一化

GRPO 还采用一个似乎继承自早期 LLM-PPO 的细节：序列归一化。将回答对数概率展开为逐时间步之和，当前估计为：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^{B} \sum_{j=1}^{G} \frac{r\left(y^{(i,j)} \mid x^{(i)}\right) - \mu_i}{\mathrm{std}_i} \nabla_\theta \sum_{t=1}^{\mathrm{len}\left(y^{(i,j)}\right)} \log \pi_\theta\left(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}\right). \tag{26}$$

原文第 12 页

再加入序列长度归一化：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^{B} \sum_{j=1}^{G} \frac{r\left(y^{(i,j)} \mid x^{(i)}\right) - \mu_i}{\mathrm{std}_i} \nabla_\theta \left(\frac{1}{\mathrm{len}\left(y^{(i,j)}\right)} \sum_{t=1}^{\mathrm{len}\left(y^{(i,j)}\right)} \log \pi_\theta\left(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}\right) \right). \tag{27}$$

原文第 12 页

这进一步改变期望：不再对跨序列的所有 token 等权，而是相对降低长序列 token 的权重。下一节通过消融检验这一选择。目前先按原论文实现标准 GRPO，因此保留序列归一化。

4.1.8 汇总

尚有一个 GRPO 组成部分：带裁剪的重要性重加权，将在异策略 RL 部分介绍。同策略每个推理批次只做一次梯度更新；异策略为加速，会对同一批次做多次更新。从第二次起，样本来自旧模型，已相对当前策略过时，这就是 off-policy。重要性重加权使估计器能利用这些旧样本。目前只做同策略，因此尚不需要它。

同策略 GRPO 的完整算法如下：

Algorithm 1: On-policy Group Relative Policy Optimization (GRPO)

Input initial policy model π_{θ_0}; reward function r; task distribution (or dataset) ρ; learning rate α

Output π_θ

policy model $\pi_\theta \leftarrow \pi_{\theta_0}$

For step $= 1, \ldots,$ `n_grpo_steps`

Sample a batch of questions $x^{(1)}, \ldots, x^{(B)} \overset{\text{iid}}{\sim} \rho$

Sample G outputs per question: $y^{(i,1)}, \ldots, y^{(i,G)} \overset{\text{iid}}{\sim} \pi_\theta\big(y \mid x^{(i)}\big)$

Compute the on-policy GRPO policy gradient estimator

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^{B} \sum_{j=1}^{G} \frac{1}{\operatorname{len}\big(y^{(i,j)}\big)} \sum_{t=1}^{\operatorname{len}\left(y^{(i,j)}\right)} \frac{r\big(y^{(i,j)} \mid x^{(i)}\big) - \mu_i}{\operatorname{std}_i} \nabla_\theta \log \pi_\theta\Big(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}\Big), \tag{28}$$

where

$$\mu_i = \frac{1}{G} \sum_{j=1}^{G} r\big(y^{(i,j)} \mid x^{(i)}\big) \tag{29}$$

denotes the group mean and

$$\operatorname{std}_i = \sqrt{\frac{1}{G} \sum_{j=1}^{G} \big(r\big(y^{(i,j)} \mid x^{(i)}\big) - \mu_i\big)^2} \tag{30}$$

denotes the group standard deviation.

Update θ with your favorite optimizer (we provide the stochastic gradient ascent update below):

$$\theta \leftarrow \theta + \alpha \hat{g}. \tag{31}$$

原文第 13 页

算法 1 中文说明：输入初始策略 π_θ0、奖励函数 r、任务分布 / 数据集 ρ、学习率 α，输出训练后策略。第 1 行用初始策略初始化；第 2-4 行循环 n_grpo_steps 次，每次采样 B 个问题、每题采样 G 个回答；第 5 行按式 (28)-(30)，减组均值、除组标准差，并按序列长度归一化，计算策略梯度；第 6 行用所选优化器更新参数，式 (31) 给出随机梯度上升形式。

4.2 实现同策略 GRPO

4.2.1 使用 Hugging Face 模型

此前使用 cs336_basics 中自己的模型，本作业直接用 transformers 加载预训练模型。也可自己实现 Transformer 和权重加载器，但架构必须与 OLMo-2-0425-1B 一致。

以下起始代码位于 cs336_alignment/checkpoint.py，以 bfloat16 和 FlashAttention-2 加载，节省内存：

```python
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

def get_model_and_tokenizer(model_id_or_dir: str, device: str):
    model = AutoModelForCausalLM.from_pretrained(
        model_id_or_dir,
        device_map=device,
        torch_dtype=torch.bfloat16,
        attn_implementation="eager" if device=='cpu' else "flash_attention_2",
    )
```

原文第 13 页

```python
    tokenizer = AutoTokenizer.from_pretrained(model_id_or_dir)
    return model, tokenizer
```

原文第 14 页

model_id_or_directory 可以是 allenai/OLMo-2-0425-1B 等名称，也可以是目录路径；目录通常由 save_pretrained 生成。保存训练结果可调用：

```python
# Save the model weights
model.save_pretrained(save_directory=output_dir)
tokenizer.save_pretrained(save_directory=output_dir)
```

原文第 14 页

先实现分词辅助函数。提示和回答分别分词，不添加特殊 token，再直接拼接 prompt_ids + response_ids；中间不插入 EOS、BOS 或分隔符。

还要构造与移位后的 labels 对齐的布尔 response_mask：标签 token 来自回答时为 True，提示与 padding 位置为 False。训练中用它只对回答 token 计算损失。

题目（tokenize_prompt_and_output）：提示与输出分词（1 分）

提交内容：实现 tokenize_prompt_and_output，分别分词、无特殊标记拼接，并生成 response_mask。建议接口：

```python
def tokenize_prompt_and_output(
    prompt_strs: list[str],
    output_strs: list[str],
    tokenizer: PreTrainedTokenizer,
) -> dict[str, torch.Tensor]:
```

原文第 14 页

参数：prompt_strs 为提示字符串列表；output_strs 为输出字符串列表；tokenizer 为所用 PreTrainedTokenizer。

返回 dict[str,torch.Tensor]。令 prompt_and_output_lens 为各拼接序列长度列表，字典包含：

• input_ids：形状 (batch_size,max(prompt_and_output_lens)-1)，拼接后的 token 序列去掉最后一个 token。

• labels：相同形状，为移位输入，即去掉第一个 token 的序列。

• response_mask：相同形状，与 labels 对齐，标签属于回答时为 1，提示和 padding 为 0。

实现 adapters.run_tokenize_prompt_and_output，运行 uv run pytest -k test_tokenize_prompt_and_output 并确保通过。

分词后可这样运行模型：

```
input_ids = train_batch["input_ids"].to(device)
labels = train_batch["labels"].to(device)
logits = model(input_ids).logits
```

原文第 15 页

接着实现计算回答逐 token 对数概率的原语，供策略梯度使用。RL 中也常记录逐 token 熵，因此还要实现 return_token_entropy 选项。

题目（get_response_log_probs）：回答对数概率与熵（1 分）

提交内容：实现 get_response_log_probs，从因果语言模型得到以此前 token 为条件的逐 token 对数概率，并可选返回下一 token 分布的熵。建议接口：

```
def get_response_log_probs(
    model: PreTrainedModel,
    input_ids: torch.Tensor,
    labels: torch.Tensor,
    return_token_entropy: bool = False,
) -> dict[str, torch.Tensor]:
```

原文第 15 页

参数：model 为 Hugging Face PreTrainedModel，应放在正确设备；若不需要梯度，应处于推理模式。input_ids 为 (batch_size,sequence_length) 的提示加回答；labels 为同形状的移位标签；return_token_entropy=True 时也返回熵。

返回字典："log_probs" 形状 (batch_size,sequence_length)，值为条件对数概率 log p_θ(x_t|x_<t)；可选 "token_entropy" 同形状，表示各位置下一 token 分布的熵，只在 return_token_entropy=True 时出现。

实现 adapters.run_get_response_log_probs，通过 uv run pytest -k test_get_response_log_probs。

4.2.2 在强化学习循环中使用 vLLM

RL 需要生成 rollout。本作业把 Hugging Face 模型与优化器放在一张 GPU 上训练，把含模型与 KV cache 的 vLLM 放在另一张 GPU 上推理。因此除初始化和生成外，每次推理前还需同步两设备的权重。cs336_alignment/vllm_utils.py 提供：

```python
@dataclass
class VLLMServer:
    gpu: int = 1 # Run training on gpu 0 and inference on gpu 1

    # Create the NCCL weight-transfer group between the training GPU and vLLM.
    def init_weight_sync(self, policy_device: str): ...

    # Copy the current Hugging Face policy weights into the vLLM server and
    # reset vLLM caches that depended on the old weights.
    def sync_policy_weights(self, policy: torch.nn.Module) -> None: ...

    # Generate rollouts from the current vLLM weights.
    def generate_completions(
        self,
        prompts: list[str],
        sampling_params: dict,
        batch_size: int | None = None,
    ) -> list[VLLMCompletion]: ...
```

原文第 16 页

代码注释含义：GPU 0 训练、GPU 1 推理；init_weight_sync 创建训练 GPU 与 vLLM 的 NCCL 权重传输组；sync_policy_weights 复制当前策略权重并清空依赖旧权重的缓存；generate_completions 用当前 vLLM 权重生成 rollout。

同前面的提示实验，temperature=1.0、top-p=1.0、最大生成长度 512。提示要求以 </answer> 结束，因此可配置：

```python
# Based on Dr. GRPO: stop when the model completes its answer
# https://github.com/sail-sg/understand-r1-zero/blob/
#   c18804602b85da9e88b4aeeb6c43e2f08c594fbc/train_zero_math.py#L167
sampling_params['stop'] = ["</answer>"]
sampling_params['include_stop_str_in_output'] = True
```

原文第 16 页

4.2.3 GRPO 组件

接下来实现 GRPO 损失各组件。后面要支持不同优势归一化和重要性重加权，因此接口设计成可通过参数切换变体、尽量复用代码。目前只需标准 GRPO。

先实现回答奖励计算。

题目（compute_rollout_rewards）：计算 rollout 奖励（1 分）

提交内容：实现 compute_rollout_rewards，计算各回答的原始奖励。建议接口：

```python
def compute_rollout_rewards(
    reward_fn: Callable[[str, str], dict[str, float]],
    rollout_responses: list[str],
    repeated_ground_truths: list[str],
) -> tuple[torch.Tensor, dict[str, float]]:
```

原文第 17 页

参数：reward_fn 为 Callable[[str,str],dict[str,float]]，对照标准答案评分，返回含 "reward"、"format_reward"、"answer_reward" 的字典；rollout_responses 是策略生成回答列表，长度 rollout_batch_size=n_prompts_per_rollout_batch*group_size；repeated_ground_truths 是同长度标准答案列表，每题标准答案重复 group_size 次。

返回 tuple[torch.Tensor,dict[str,float]]：raw_rewards 为形状 (rollout_batch_size,) 的未归一化奖励；metadata 为日志统计，至少包括该 rollout 批次的总奖励均值和格式奖励均值。

实现 adapters.run_compute_rollout_rewards，通过 uv run pytest -k compute_rollout_rewards。

接着将奖励按组归一化为优势。输入奖励已展平，需要 group_size 恢复组结构。通过测试需支持 baseline="mean"、advantage_normalizer="std"；除法前在组标准差上加 advantage_eps，避免除零。

题目（compute_group_normalized_rewards_grpo）：组归一化（1 分）

提交内容：实现 compute_group_normalized_rewards，按组归一化原始奖励，返回优势与有用的元数据。当前只需支持 baseline="mean" 和 advantage_normalizer="std"，其他输入可抛 NotImplementedError；后面再补全。归一化分母记得加 advantage_eps。

```
def compute_group_normalized_rewards(
    raw_rewards: torch.Tensor,
    group_size: int,
    baseline: Literal["mean", "none"] = "mean",
    advantage_eps: float = 1e-6,
    advantage_normalizer: Literal["std", "none", "mean"] = "std",
):
```

原文第 18 页

参数：raw_rewards 为 (rollout_batch_size,) 原始奖励；group_size 为每题回答数；baseline 可选 "mean"、"none"，本题实现 mean，即减组均值，后续 none 表示不减基线；advantage_eps 为防止除零的小常数；advantage_normalizer 可选 "std"、"none"、"mean"，本题 std 表示除组标准差，后续 none 不归一化，mean 除组平均奖励。

返回 tuple[torch.Tensor,dict[str,float]]：advantages 为形状 (rollout_batch_size,) 的组归一化奖励；metadata 可记录奖励均值、标准差、最大最小值等。

实现 adapters.run_compute_group_normalized_rewards，通过 uv run pytest -k compute_group_normalized_rewards_grpo。

下一步计算逐 token 策略梯度损失。算法 1 写的是梯度估计，本题要写一个对其求导后得到相应估计项的损失，再跨 token 聚合。它并非传统意义上期待在训练中持续下降的损失，只是用于产生所需梯度的表达式。

具体目标为：

$$J_\theta^{\text{GRPO-on-policy}} = \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{1}{\text{len}(y^{(i,j)})}\sum_{t=1}^{\text{len}(y^{(i,j)})}\frac{r(y^{(i,j)} \mid x^{(i)}) - \mu_i}{\text{std}_i}\log \pi_\theta\left(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}\right), \quad (32)$$

原文第 18 页

对其求导得到算法 1 的梯度。PyTorch 优化器做梯度下降，因此实现应返回该目标的负值作为损失。compute_policy_gradient_loss 计算每条序列、每个 token 的项；aggregate_loss_across_microbatch_sequence 再跨 token 和序列聚合。

题目（compute_policy_gradient_loss_on_policy）：同策略策略梯度损失（1 分）

交付内容：实现 compute_policy_gradient_loss，根据原始奖励或预先算好的优势，计算逐 token 策略梯度损失。本节只需支持同策略情形，即 importance_reweighting_method="none"；暂时忽略旧策略的对数概率与裁剪，其余选项可抛出 NotImplementedError，后续再实现。

建议接口如下：

```
def compute_policy_gradient_loss(
    raw_rewards_or_advantages: torch.Tensor,
    policy_log_probs: torch.Tensor,
    importance_reweighting_method: Literal["none", "noclip", "grpo", "gspo"] = "none",
    old_log_probs: torch.Tensor | None = None,
    cliprange: float | None = None,
    response_mask: torch.Tensor | None = None,
) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

原文第 19 页

参数：raw_rewards_or_advantages 为形状 (batch_size,) 或 (batch_size, 1) 的张量，每条 rollout 对应一个标量；policy_log_probs 为形状 (batch_size, sequence_length) 的张量，保存每个 token 的策略对数概率。

importance_reweighting_method 可选 "none"（不做重要性重加权）、"noclip"（重加权但不裁剪）、"grpo"（PPO/GRPO 风格的 token 级重加权与裁剪）或 "gspo"（GSPO 风格的序列级重加权与裁剪）。

old_log_probs 的形状为 (batch_size, sequence_length)，除 "none" 外均必须提供。cliprange 是裁剪参数 ε，使用 "grpo" 或 "gspo" 时必须提供。response_mask 是可选的 (batch_size, sequence_length) 张量；GSPO 需要它，只在回答 token 上对序列的对数概率比取平均。

返回：(1) per_token_policy_gradient_loss，形状为 (batch_size, sequence_length)，训练循环中再沿批次和序列维聚合；(2) metadata，保存裁剪比例等元数据。

实现 adapters.run_compute_policy_gradient_loss，然后运行 uv run pytest -k test_compute_policy_gradient_loss_on_policy，确保通过测试。

标准 GRPO 的损失聚合方式是先在每条序列的 token 上取平均，再对序列取平均。后面还会实现使用固定常数归一化的方式，使结果更忠实于策略梯度。

题目（aggregate_loss_across_microbatch_sequence）：按序列聚合损失（0.5 分）

交付内容：实现 aggregate_loss_across_microbatch，将逐 token 损失和掩码聚合为平均损失。本题只需支持 loss_normalization="sequence"；其他选项可以暂时抛出 NotImplementedError。

建议接口如下：

```
def aggregate_loss_across_microbatch(
    per_token_policy_gradient_loss: torch.Tensor,
    mask: torch.Tensor,
    loss_normalization: Literal["sequence", "constant"] = "sequence",
    normalization_constant: int | None = None,
) -> torch.Tensor:
```

原文第 20 页

per_token_policy_gradient_loss：形状 (batch_size, sequence_length) 的逐 token 策略梯度损失，在训练循环中沿批次和序列维聚合。mask：相同形状的张量，指明哪些位置参与损失计算。

loss_normalization："sequence" 表示先对每条序列的损失取平均，再对序列取平均；"constant" 表示将总损失除以一个常数。normalization_constant：这个除数，使用 "constant" 时必须提供。

返回 loss：包含平均损失的标量张量，必须能够在其上调用 backward。

实现 adapters.run_aggregate_loss_across_microbatch，运行 uv run pytest -k test_aggregate_loss_across_microbatch_sequence，确保通过测试。

4.2.4 GRPO 训练步骤

现在可以把这些组件拼成完整的训练步骤。

梯度累积。为了让推理阶段充分利用 GPU，我们需要较大的批量。同策略 RL 中，训练批量等于推理批量，GPU 显存通常无法容纳整个批次的梯度计算。因此要将批次分成若干微批次（microbatch），累积各微批次的梯度。难点在于正确归一化，使累积得到的梯度与一次处理整个批次的梯度相同。

PyTorch 中每个权重张量都有 .grad 属性保存梯度。调用 loss.backward() 前，它为 None；调用后，它保存相应梯度。通常接下来执行优化器更新，再用 optimizer.zero_grad() 清空梯度，为下一次迭代做准备：

```
# Forward pass.
logits = model(inputs)
loss = loss_fn(logits, labels)

# Backward pass.
loss.backward()

# Update weights.
optimizer.step()
# Zero gradients in preparation for next iteration.
optimizer.zero_grad()
```

原文第 21 页

要实现梯度累积，把批次分成 k 个微批次，只在整个批次处理完后调用一次 optimizer.step() 和 optimizer.zero_grad()。若采用序列归一化，即先在每条序列内取平均、再对序列取平均，那么算出微批次的平均损失后，还要按微批次所含序列数占整个批次的比例重加权：

```
gradient_accumulation_steps = 4
microbatch_size = len(inputs) // gradient_accumulation_steps
for i in range(0, len(inputs), microbatch_size):
    inputs_microbatch = inputs[i:i+microbatch_size]
    labels_microbatch = labels[i:i+microbatch_size]

    # Forward pass.
    logits = model(inputs_microbatch)
    loss = loss_fn(logits, labels_microbatch) * (len(inputs_microbatch) / len(inputs))

    # Backward pass.
    loss.backward()
# Update weights once across entire batch.
optimizer.step()
# Zero gradients once across entire batch.
optimizer.zero_grad()
```

原文第 21 页

之后使用常数归一化时，微批次代码也需要相应调整。

实现训练步骤。下面的函数接收模型、分词器、优化器、奖励函数、提示词、rollout 和各项超参数，累积梯度并执行一次优化器更新。必须按上述方法实现梯度累积，避免显存耗尽。在优化器更新前，将梯度范数裁剪到 max_grad_norm。返回批次训练损失和供日志记录使用的元数据。

训练步骤至少记录：损失、梯度范数、token 熵、训练奖励（总奖励与格式奖励）。也可以记录其他指标，例如 k 从 1 到 group_size 的 pass@k：前 k 条 rollout 中至少有一个正确答案的提示词占比。

题目（grpo_train_step_standard_on_policy）：GRPO 训练步骤（5 分）

交付内容：给定模型、分词器和 rollout，实现策略梯度的一次批次更新。本节只需支持标准同策略 GRPO：baseline="mean"、advantage_normalizer="std"、importance_reweighting_method="none"、loss_normalization="sequence"。不支持的输入可抛出 NotImplementedError。

建议接口如下：

```python
def grpo_train_step(
    model: PreTrainedModel,
    tokenizer: PreTrainedTokenizer,
    optimizer: Optimizer,
    gradient_accumulation_steps: int,
    max_grad_norm: float | None,
    reward_fn: Callable[[str, str], dict[str, float]],
    repeated_prompts: list[str],
    rollout_responses: list[str],
    repeated_ground_truths: list[str],
    group_size: int,
    # Reward normalization
    baseline: Literal["mean", "none"] = "mean",
    advantage_eps: float = 1e-6,
    advantage_normalizer: Literal["std", "none", "mean"] = "std",
    # Importance reweighting and clipping
    importance_reweighting_method: Literal["none", "noclip", "grpo", "gspo"] = "none",
    old_log_probs: torch.Tensor | None = None,
    cliprange: float | None = None,
    # Loss normalization
    loss_normalization: Literal["sequence", "constant"] = "sequence",
    normalization_constant: int | None = None,
) -> tuple[torch.Tensor, dict[str, torch.Tensor | float]]:
```

原文第 22 页

函数以 gradient_accumulation_steps 个微批次完成前向和反向传播。参数说明如下。

• model：待训练的 HuggingFace PreTrainedModel；tokenizer：用于分词的 PreTrainedTokenizer；optimizer：模型的优化器。

• gradient_accumulation_steps：每次优化器更新所包含的微批次数。max_grad_norm：若不为 None，在 optimizer.step() 前将梯度范数裁剪到此值。

• reward_fn：对照真实答案给 rollout 打分，返回包含 "reward"、"format_reward" 和 "answer_reward" 的字典。

• repeated_prompts：提示词列表，每个样本重复 group_size 次，因此长度为 rollout_batch_size。

• rollout_responses：策略生成的回答，列表长度为 rollout_batch_size = n_prompts_per_rollout_batch * group_size。

• repeated_ground_truths：真实答案列表，每个答案重复 group_size 次，因此长度也为 rollout_batch_size。group_size：每道问题生成的回答数，即每组大小。

• baseline："mean" 表示减去组内平均奖励，"none" 表示不处理。advantage_eps：归一化时防止除零的小常数。advantage_normalizer："std" 表示除以组内标准差，"none" 表示不处理，"mean" 表示除以组内平均奖励。

• importance_reweighting_method："none" 不做重要性重加权；"noclip" 重加权但不裁剪；"grpo" 做 PPO/GRPO 风格的 token 级重加权和裁剪；"gspo" 做 GSPO 风格的序列级重加权和裁剪。

• old_log_probs：形状为 (batch_size, sequence_length)，除 "none" 外均必须提供。cliprange：裁剪参数 ε，使用 "grpo" 或 "gspo" 时必须提供。

• loss_normalization："sequence" 先在每条序列内对损失取平均，再对序列取平均；"constant" 将总损失除以整个训练期间固定不变的常数。normalization_constant 是此除数，使用 "constant" 时必须提供。

返回值为二元组：loss 是已考虑梯度累积缩放的批次损失标量，用于日志记录；metadata 是字典，包含底层损失函数返回的统计量、裁剪前的梯度范数以及其他需要记录的统计量。

实现 adapters.run_grpo_train_step，运行 uv run pytest -k test_grpo_train_step_standard_on_policy，确保通过测试。

4.3 实验

现在可以构建完整的 GRPO 训练循环：加载模型和数据集，初始化日志（例如 Wandb）、vLLM 服务与优化器，然后运行 RL 训练循环。

下面给出建议超参数。若脚本正确，多数随机种子下的验证奖励应随训练增加。

```
n_train_examples = 6400
n_val_examples = 1024
num_rollout_steps = 200
learning_rate = 1e-5
rollout_batch_size = train_batch_size = 256
group_size = 8
gradient_accumulation_steps = 32
sampling_temperature = 1.0
sampling_max_tokens = 512
max_grad_norm = 1.0
optimizer = torch.optim.AdamW(
    policy.parameters(), lr=learning_rate, betas=(0.9, 0.95), weight_decay=0.0
)
```

原文第 24 页

注意，rollout_batch_size 和 train_batch_size 统计的是回答数，而不是提示词数。两者都为 256，意味着 32 条提示词，每条生成 8 个 rollout。

一个合理的默认设置是每 10 个 rollout 批次在验证集上评估一次。CoT/RL 评估噪声较大，至少使用 1024 个验证样本。除前述指标外，还应记录生成的 rollout，直观了解模型行为；可以每 40 个 rollout 批次记录当前批次的训练 rollout。训练与验证样本可以分别使用 data/gsm8k/train.jsonl 和 data/gsm8k/test.jsonl。

策略梯度估计的方差可能很大，而 RL 又有自我强化效应，因此不同训练运行的轨迹常有很大差异。确认脚本正确后，请使用 4 个随机种子运行 RL 循环。如果固定种子不能保证完全复现，也可以直接运行实验 4 次。图中必须体现不同运行之间的波动，不能只画均值。可采用以下方式：

- 绘制 4 次运行的均值，同时画出各次运行的曲线。为保证可读性，可将单次运行另画一张图，或使用较低透明度、较细线条、虚线。
- 绘制均值及置信区间阴影。在每个步骤计算样本均值 μ 和样本标准差 σ；一个合理的区间是 $\mu \pm 1.96 \times \sigma/\sqrt{n}$，其中 n 是种子数。
- 绘制均值，以及每一步的最小值和最大值。

题目（grpo_experiments_standard_on_policy）：用 GRPO 提升 OLMo-2-0425-1B 在 GSM8K 上的表现（2 B200 小时，10 分）

(a) 编写 GRPO 训练循环脚本，接收模型名称、提示词、训练集路径、验证集路径、采样超参数和训练超参数。首先初始化 vLLM 服务、wandb 日志、数据集、模型及优化器；随后循环执行：与 vLLM 服务同步权重、生成训练 rollout、在训练批次上进行策略梯度更新，并定期评估验证集表现，同时记录生成结果。

交付内容：在 GSM8K 上对 OLMo-2-0425-1B 运行标准同策略 GRPO 的脚本。

(b) 先运行约 50 步，确认验证奖励提高，且 rollout 随训练变得合理。奖励一开始接近零是正常现象；视随机种子而定，可能需要几步才能采到非零奖励的 rollout，模型才开始改善。

交付内容：能够支持脚本正确性的证据，例如验证奖励随步骤提高、rollout 内容合理。

(c) 确认脚本正确后，使用上面给出的超参数和 4 个随机种子，在 GSM8K 上训练 OLMo-2-0425-1B，提示词使用零样本 r1_zero。正确实现应使验证奖励随训练改善。

记录并绘制以下指标随时间的变化，同时体现运行之间的方差，例如均值/标准差或最小值/最大值：损失、梯度范数、token 熵、训练奖励（总奖励与格式奖励）、验证奖励（总奖励与格式奖励）、验证集平均回答长度，以及其他有助于调试的指标。还要定期记录和观察 rollout：训练是否改善了回答？

交付内容：每项指标一张图，并用几句话说明其训练趋势和不同运行之间的波动。

交付内容：若干训练前后模型 rollout 的例子。

交付内容：一种训练流程，其最终验证准确率在随机种子之间取平均后至少达到 25%。

有了可用脚本，就可以调超参数并做消融实验。先调通常对训练影响最大的学习率。

题目（grpo_learning_rate）：调节学习率（4 B200 小时，3 分）

从建议超参数出发，扫描多个学习率，至少包含一个低于默认值和一个高于默认值的学习率。报告最终验证奖励；若优化器发散，也需注明。根据上一题观察到的方差，决定每个设置使用多少个随机种子。后续作业中可以使用调好的学习率。

交付内容：最终验证奖励随学习率变化的图，以及几句说明。

提示词也会影响 RL 动态：RL 会提高模型生成的某些 rollout 的权重，因此提示词会塑造训练中探索到的 rollout。下面考察这种影响。

题目（grpo_prompt_ablation）：提示词消融（4 B200 小时，3 分）

使用 question_only 和 r1_zero_three_shot 提示词，各用几个随机种子运行 GRPO。与前面的零样本 r1_zero 相比如何？哪个提示词的平均奖励最高、方差最低？其他日志指标是否存在系统差异？结合运行间的方差，你对结论有多大把握？

交付内容：分析说明，并用所引用指标的图表支持结论。

5 RL 算法变体

GRPO 是语言模型 RL 的常用算法，但许多论文对其算法选择提出了不同意见。本节学习这些选择背后的理论，并通过受控实验观察哪种算法更适合当前模型与数据集。本节研究同策略设置，下一节研究异策略算法。

5.1 Dr. GRPO

推导 GRPO 时，有两个选择使梯度估计不再具有“正确”的期望 ∇θJθ，其中 Jθ 是模型的期望奖励：用标准差归一化优势，以及按序列长度归一化损失。Dr. GRPO 论文 [Z. Liu 等，2025] 主张撤销这两个选择：不再除以标准差，也不再先将每条序列按长度归一化，而是把总损失除以固定常数 Z。梯度估计器如下：

$$\hat{g} \leftarrow \frac{1}{Z}\sum_{i=1}^{B}\sum_{j=1}^{G}\sum_{t=1}^{\operatorname{len}(y^{(i,j)})}\left(r\left(y^{(i,j)} \mid x^{(i)}\right)-\mu_i\right)\nabla_\theta \log \pi_\theta\left(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}\right), \tag{33}$$

原文第 26 页

Z 通常设为训练批量大小乘以最大生成长度，即 Z=BGL，其中 L 是最大生成长度，本作业为 512。

题目（think_about_length_normalization）：思考长度归一化（1 分）

在运行实验前，比较“每条序列分别除以自身长度”和“所有序列使用同一常数归一化”两种方式。各有什么优缺点？有哪些具体设置或例子更适合其中一种方式？

交付内容：几句讨论。

题目（compute_group_normalized_rewards_drgrpo）：Dr. GRPO 组归一化（0.5 分）

交付内容：扩展 compute_group_normalized_rewards，支持 advantage_normalizer="none"。虽然 Dr. GRPO 使用 baseline="mean"，本题还要求支持 baseline="none"。

实现 adapters.run_compute_group_normalized_rewards，运行 uv run pytest -k compute_group_normalized_rewards_drgrpo，确保通过测试。

题目（aggregate_loss_across_microbatch_constant）：Dr. GRPO 损失聚合（0.5 分）

交付内容：扩展 aggregate_loss_across_microbatch，支持 loss_normalization="constant"。

实现 adapters.run_aggregate_loss_across_microbatch，运行 uv run pytest -k test_aggregate_loss_across_microbatch_constant，确保通过测试。

5.2 拒绝采样微调

接下来考察更简单的算法，通常称为拒绝采样微调（rejection fine tuning，RFT）或专家迭代（expert iteration，EI）：生成许多 rollout，只保留正确的，再对这些正确 rollout 做监督微调，即最小化下一词预测的对数损失。其梯度为：

$$\hat{g} \leftarrow \nabla_\theta \left[\frac{1}{Z} \sum_{i=1}^{B} \sum_{j=1}^{G} \mathbb{1}\{r(y^{(i,j)} \mid x^{(i)}) = 1\} \sum_{t=1}^{\text{len}(y^{(i,j)})} \log \pi_\theta\left(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}\right) \right], \tag{34}$$

原文第 27 页

Z 是与 Dr. GRPO 中类似的常数归一化因子；指示函数只保留奖励为 1 的正确回答。下一题将思考：它是否也是优化期望奖励的策略梯度，以及它与 GRPO 有何关系。实现时同样返回目标的负值，使梯度下降实现目标上的梯度上升。

题目（think_about_rft）：思考 RFT（2 分）

采用常数归一化的 RFT 目标为：

$$J_\theta = \frac{1}{Z} \sum_{x} \sum_{j=1}^{G} \mathbb{1}\{r(y^{(j)} \mid x) = 1\} \log \pi_\theta(y^{(j)} \mid x), \tag{35}$$

原文第 28 页

x 为提示词，y^(j) 独立同分布地采样自 πθ，G 为每道问题的生成数，Z 为归一化常数，r 为奖励函数。其梯度估计为 ∇θJθ。作为对照，Dr. GRPO 的同策略梯度估计器使用常数归一化且不做标准差归一化：

$$\frac{1}{Z} \sum_{x} \sum_{j=1}^{G} (r(y^{(j)} \mid x) - \mu) \nabla_\theta \log \pi_\theta(y^{(j)} \mid x), \tag{36}$$

原文第 28 页

其中 μ 为组内平均奖励。假设奖励为二值，比较两者：它们的期望是否相同？哪个方差更低？什么情况下你更倾向于使用其中一种？

交付内容：几句讨论。

5.3 MaxRL

最大似然强化学习（Maximum Likelihood Reinforcement Learning，MaxRL）[F. Tajwar 等，2026] 将优势除以组内平均奖励，而不是像 GRPO 那样除以标准差，也不像 Dr. GRPO 那样不做优势归一化。梯度估计为：

$$\hat{g} \leftarrow \frac{1}{Z} \sum_{i=1}^{B} \sum_{j=1}^{G} \sum_{t=1}^{\text{len}(y^{(i,j)})} \frac{r(y^{(i,j)} \mid x^{(i)}) - \mu_i}{\mu_i} \nabla_\theta \log \pi_\theta\left(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}\right). \tag{37}$$

原文第 28 页

原始 MaxRL 实现用批次内 token 总数 $\Sigma_i \Sigma_j$ len(y^(i,j)) 归一化，因此归一化因子依赖批次。本作业使用固定 Z，以单独考察“除以均值”和“除以标准差”的影响。

直观上，除以平均奖励 μ_i 会提高困难提示词（平均奖励较低）的权重。为形式化这一点，定义重加权后的期望奖励：

$$J_{\theta,w} = E_{x \sim \rho}\left[w(x, \text{stopgrad}(\pi_\theta)) E_{y \sim \pi_\theta} r(y \mid x) \right], \tag{38}$$

原文第 28 页

权重函数 w 依赖提示词难度 η(x)=E[y~πθ]r(y|x)。stopgrad 表示不对这个权重求导。下一题推导不同优势归一化方法诱导的权重。

题目（derive_difficulty_reweightings）：推导不同优势归一化方式诱导的难度权重（6 分）

对提示词整体取期望，标准策略梯度为：

$$\nabla_\theta J_\theta = \nabla_\theta E_{x\sim\rho}\Big[E_{y\sim\pi_\theta} r(y \mid x)\Big], \tag{39}$$

原文第 29 页

ρ 是提示词 x 的分布，πθ 是策略，r(y|x) 表示 y 是否正确回答 x。下面推导各 GRPO 变体优化的替代目标，其形式为：

$$\nabla_\theta J_{\theta,w} = \nabla_\theta E_{x\sim\rho}\Big[w(x, \mathrm{stopgrad}(\pi_\theta)) E_{y\sim\pi_\theta} r(y \mid x)\Big] \tag{40}$$

原文第 29 页

其中 w 是提示词的重加权函数，求导时不对 w 求导。

(a) Dr. GRPO 的策略梯度估计器为：

$$E_{x\sim\rho}\left[\frac{1}{Z}\sum_{j=1}^{G}\big(r(y^{(j)} \mid x) - \mu\big)\nabla_\theta \log \pi_\theta(y^{(j)} \mid x)\right] \tag{41}$$

原文第 29 页

其中 μ=(1/G)Σ$_j$r(y^(j)|x)。令 Z=G，并取 G→∞。对于什么权重函数 w，该估计器等价于优化替代目标 Jθ,w？

交付内容：用题目参数中的部分参数写出表达式，并用几句话说明理由。

(b) 假设使用常数归一化，GRPO 与 Dr. GRPO 的区别是再除以组内标准差：

$$E_{x\sim\rho}\left[\frac{1}{Z}\sum_{j=1}^{G}\frac{r(y^{(j)} \mid x) - \mu}{\mathrm{std}}\nabla_\theta \log \pi_\theta(y^{(j)} \mid x)\right], \tag{42}$$

原文第 29 页

令 Z=G，并取 G→∞。对于什么 w，该估计器等价于优化 Jθ,w？

交付内容：用题目参数中的部分参数写出表达式，并用几句话说明理由。

(c) MaxRL 改为除以组内均值：

$$E_{x\sim\rho}\left[\frac{1}{Z}\sum_{j=1}^{G}\frac{r(y^{(j)} \mid x) - \mu}{\mu}\nabla_\theta \log \pi_\theta(y^{(j)} \mid x)\right]. \tag{43}$$

原文第 29 页

令 Z=G，并取 G→∞。对于什么 w，该估计器等价于优化 Jθ,w？

交付内容：用题目参数中的部分参数写出表达式，并用几句话说明理由。

题目（think_about_advantage_normalization）：思考优势归一化（2 分）

在运行实验前，比较“除以组内标准差”“除以组内均值”和“不做优势归一化”。各有什么优缺点？有哪些具体设置或例子更适合其中某种方式？

交付内容：几句讨论。

题目（compute_group_normalized_rewards_maxrl）：MaxRL 组归一化（0.5 分）

交付内容：扩展 compute_group_normalized_rewards，支持 advantage_normalizer="mean"。与 "std" 一样，在分母中加上 advantage_eps，避免除零。

实现 adapters.run_compute_group_normalized_rewards，运行 uv run pytest -k compute_group_normalized_rewards_maxrl，确保通过测试。

5.4 实验

现在可以比较这些策略梯度估计器。先扩展 grpo_train_step，支持前面实现的组件组合：

• GRPO_constant：baseline="mean"，advantage_normalizer="std"，loss_normalization="constant"。

• Dr_GRPO：baseline="mean"，advantage_normalizer="none"，loss_normalization="constant"。

• RFT：baseline="none"，advantage_normalizer="none"，loss_normalization="constant"。

• MaxRL（常数归一化）：baseline="mean"，advantage_normalizer="mean"，loss_normalization="constant"。

为了加速训练，算出每条序列的归一化优势后，可以不把优势为零的序列送入模型，因为它们对梯度没有贡献。二值奖励下有两种情况：baseline="mean" 时，同组所有序列奖励相同，则它们的优势全为零；baseline="none" 时，奖励为零的序列优势也为零。

去掉这些序列后，可以同比例减少 gradient_accumulation_steps，保持 microbatch_size 不变。例如一半序列优势为零，就可以将累积步数从 k 减为 k/2。这对零奖励序列较多的 RFT 尤其有用。但务必检查数学和实现，保证剪除前后的梯度完全相同。

题目（grpo_train_step_variants_on_policy）：GRPO 同策略训练步骤变体（2.5 分）

交付内容：扩展 grpo_train_step，支持所有同策略变体，importance_reweighting_method 仍为 "none"。需支持 baseline∈{"mean","none"}、advantage_normalizer∈{"std","none","mean"}、loss_normalization∈{"sequence","constant"}。为加速训练，避免将零优势序列送入模型。

baseline="none" 时，错误样本奖励为零，在损失中的权重也为零，因此无需经过待训练模型；请利用这一点加速实现。

实现 adapters.run_grpo_train_step，运行 uv run pytest -k test_grpo_train_step_variants_on_policy，确保通过测试。

实现训练步骤后，用与之前相同的超参数比较这些方法是否优于标准 GRPO。

关于调参：要判断某方法是否优于另一方法，最严谨的做法是分别为每种方法调参。但本作业计算资源有限，因此采用较便宜的办法：为基线调参，再把这些参数用于新方法。这里即采用在标准 GRPO 上调好的学习率。如果

新方法在这种条件下胜过基线，仍可以认为它更好，因为未充分调参的表现只是它充分调参后表现的下界；反之，若新方法较差，则不能据此断言它劣于基线，因为我们没有为它充分调参。

题目（grpo_experiments_variants_on_policy）：比较不同 RL 算法的表现（8 B200 小时，10 分）

保持与标准 GRPO 运行相同的超参数，学习率采用你选定的值；每种变体运行 4 个随机种子，使用零样本 r1_zero 提示词。

• GRPO_constant：标准 GRPO，但聚合损失时使用常数归一化，替代序列归一化。

• Dr_GRPO：在 GRPO_constant 基础上设置 advantage_normalizer="none"。

• RFT：在 GRPO_constant 基础上设置 advantage_normalizer="none"、baseline="none"。无需将错误样本送入待训练模型，应能加快训练。

• MaxRL：在 GRPO_constant 基础上设置 advantage_normalizer="mean"。原始 MaxRL 按微批次 token 数归一化，本作业改用常数归一化。

与前面的标准 GRPO 相比，各方法表现如何？哪个平均表现最好，哪个方差最低？哪些方法可能受益于进一步调参？结合运行之间的方差，你对发现有多大把握？

交付内容：分析说明，并用所引用指标的图表支持结论。

6 异策略 RL

此前的算法和实验都是同策略的：每个梯度估计使用的样本直接来自正被更新的模型，即每个推理批次只执行一次训练更新，训练批量等于推理批量。本节探索异策略 RL：每个推理批次执行多次训练更新，有望加快训练，但可能降低稳定性，并增加算法复杂度。

6.1 重要性重加权

最基本的策略梯度先生成一批回答，再对它们做一次大批量梯度更新。若将这批数据划成多个小批次，每个小批次做一次更新，也许可以更快训练。这就是异策略 RL。

第一次小批次更新之后，当前策略就与生成样本的策略不同了。这些样本已经“过时”，不再来自我们要计算策略梯度的当前策略。记推理策略为 π_o，当前策略为 $\pi\theta$，有：

$$E_{x\sim\rho}E_{y\sim\pi_0(y\mid x)}[r(y\mid x)\nabla_\theta\log\pi_\theta(y\mid x)]\neq E_{x\sim\rho}E_{y\sim\pi_\theta(y\mid x)}[r(y\mid x)\nabla_\theta\log\pi_\theta(y\mid x)]. \tag{44}$$

原文第 32 页

左边是朴素异策略估计器：假装回答 y 来自当前策略，机械地沿用同策略梯度计算。这个式子说明它的期望不正确。

一种修正偏差的办法是重要性重加权，得到：

$$E_{x\sim\rho}E_{y\sim\pi_0(y\mid x)}\left[\frac{\pi_\theta(y\mid x)}{\pi_0(y\mid x)}r(y\mid x)\nabla_\theta\log\pi_\theta(y\mid x)\right], \tag{45}$$

原文第 32 页

权重 πθ(y|x)/π₀(y|x) 会提高仍与当前策略相关的样本权重，并降低当前策略赋予低概率的过时样本权重。不难验证，修正后的期望正确：

$$E_{x\sim\rho}E_{y\sim\pi_0(y\mid x)}\left[\frac{\pi_\theta(y\mid x)}{\pi_0(y\mid x)}r(y\mid x)\nabla_\theta\log\pi_\theta(y\mid x)\right]=E_{x\sim\rho}E_{y\sim\pi_\theta(y\mid x)}[r(y\mid x)\nabla_\theta\log\pi_\theta(y\mid x)]. \quad (46)$$

原文第 32 页

代价是估计器方差增加。如果重要性权重可以大到 C，方差也可能放大 C 倍。对语言模型，序列权重是 len(y) 个项的乘积，随回答长度呈指数变化，因此朴素的重要性重加权难以直接使用：

$$\frac{\pi_\theta(y\mid x)}{\pi_0(y\mid x)}=\prod_{t=1}^{\mathrm{len}(y)}\frac{\pi_\theta(y_t\mid x,y_{<t})}{\pi_0(y_t\mid x,y_{<t})}. \quad (47)$$

原文第 32 页

不加权的朴素异策略估计器偏差较高，但没有重加权额外引入的方差；完整重要性重加权则零偏差但高方差。下面的方法介于两者之间，用不同启发式处理偏差—方差权衡。

6.2 PPO/GRPO 风格的重要性重加权与裁剪

6.2.1 token 级重加权

PPO [J. Schulman 等，2017] 首先提出、GRPO [Z. Shao 等，2024] 随后采用的标准方法，是使用 token 级重加权代替序列级重加权。“不裁剪的 token 级重加权”估计器为：

$$E_{x\sim\rho}E_{y\sim\pi_0(y\mid x)}\left[r(y\mid x)\sum_{t=1}^{\mathrm{len}(y)}\frac{\pi_\theta(y_t\mid x,y_{<t})}{\pi_0(y_t\mid x,y_{<t})}\nabla_\theta\log\pi_\theta(y_t\mid x,y_{<t})\right]. \quad (48)$$

原文第 33 页

它不同于前面理论上完整的序列级估计器。后者按时间步展开为：

$$E_{x\sim\rho}E_{y\sim\pi_0(y\mid x)}\left[r(y\mid x)\left(\prod_{t=1}^{\mathrm{len}(y)}\frac{\pi_\theta(y_t\mid x,y_{<t})}{\pi_0(y_t\mid x,y_{<t})}\right)\sum_{t=1}^{\mathrm{len}(y)}\nabla_\theta\log\pi_\theta(y_t\mid x,y_{<t})\right]. \quad (49)$$

原文第 33 页

新式子的重加权项不再随 len(y) 指数变化，方差因而显著下降。

为理解它，推导其优化的替代目标 [T. Degris 等，2013]。记 $\tilde{\pi}_t$ 为这样一种替代策略：除第 t 步从当前策略 πθ 采样外，其余时间步都从旧策略 π₀ 采样：

$$\tilde{\pi}_t(y\mid x)=\left(\prod_{s=1}^{t-1}\pi_0(y_s\mid x,y_{<s})\right)\pi_\theta(y_t\mid x,y_{<t})\left(\prod_{s=t+1}^{\mathrm{len}(y)}\pi_0(y_s\mid x,y_{<s})\right). \quad (50)$$

原文第 33 页

假设所有回答长度均为 L，则 token 级重加权优化的是这些 $\tilde{\pi}_t$ 下期望奖励之和：

$$J_\theta^{\text{token}} = E_x \left[\sum_{t=1}^{L} E_{y \sim \tilde{\pi}_t(y \mid x)} [r(y \mid x)] \right]. \tag{51}$$

原文第 33 页

用重要性重加权改写该目标，立即可得：

$$\nabla_\theta J_\theta^{\text{token}} = \nabla_\theta E_x \left[\sum_{t=1}^{L} E_{y \sim \tilde{\pi}_t(y \mid x)} [r(y \mid x)] \right] \tag{52}$$

$$= \nabla_\theta E_x \left[\sum_{t=1}^{L} E_{y \sim \pi_0(y \mid x)} \left[\frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_0(y_t \mid x, y_{<t})} r(y \mid x) \right] \right] \tag{53}$$

$$= E_x \left[E_{y \sim \pi_0(y \mid x)} \left[\sum_{t=1}^{L} \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_0(y_t \mid x, y_{<t})} r(y \mid x) \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t}) \right] \right], \tag{54}$$

原文第 33 页

最后一步使用了本作业多次用到的对数导数技巧。

这个替代目标揭示了 token 级重加权引入的偏差。首先，第 t 项的前缀 y<t 来自 π_0，而不是 πθ，因此可能不代表当前模型的典型前缀。其次，当前策略采出 y_t 后，后缀 y>t 仍来自 π_0，因此评估动作 y_t 对未来的影响时，依据的是旧策略行为，而不是当前策略行为。序列级重加权会正确调整前缀和后缀权重，从而修正这两种偏差，但方差更高。直观上，πθ 离 π_0 越远，这些偏差越严重。

题目（derive_surrogate_objectives）：推导重要性重加权方法的替代目标（2 分）

前面看到，token 级重加权优化的是一组替代策略下的期望奖励：每个替代策略仅在一个位置使用当前策略，其余位置使用旧策略。现在考虑“成对”的重要性重加权，策略梯度估计器如下：

$$\sum_{t=1}^{\frac{L}{2}} \frac{\pi_\theta(y_{2t-1} \mid x, y_{<2t-1}) \pi_\theta(y_{2t} \mid x, y_{<2t})}{\pi_0(y_{2t-1} \mid x, y_{<2t-1}) \pi_0(y_{2t} \mid x, y_{<2t})} r(y \mid x) \nabla_\theta [\log(\pi_\theta(y_{2t-1} \mid x, y_{<2t-1}) \pi_\theta(y_{2t} \mid x, y_{<2t}))] \tag{55}$$

原文第 34 页

πθ 是当前策略，π_0 是过时的采样策略，且 y~π_0(y|x)。该估计器优化的替代目标是什么？

交付内容：用题目参数中的部分参数写出表达式，并给出推导。

6.2.2 裁剪

PPO 和 GRPO 的另一个启发式是裁剪重要性权重。πθ 离 π_0 越远，重加权的方差越大，token 级重加权的偏差也越大。因此要保持异策略训练稳定，就应使当前策略接近旧策略。

最简单的办法是减少每个推理批次的训练更新次数，但不同推理批次可承受的更新次数不同。为了尽可能提高效率，我们希望在一个推理批次上做尽量多的有效更新。一类方法通过裁剪过大或过小的重要性权重来实现这一点。

近期论文对裁剪细节意见不一。本作业采用 PPO 提出、GRPO 沿用且目前较常见的方法。将 token 级重加权与标准 GRPO 目标结合，得到：

$$J_\theta^{\text{GRPO-off-policy-noclip}} = \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{1}{\text{len}\left(y^{(i,j)}\right)}\frac{r\left(y^{(i,j)} \mid x^{(i)}\right)-\mu_i}{\text{std}_i}\sum_{t=1}^{\text{len}\left(y^{(i,j)}\right)}\frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_0(y_t \mid x, y_{<t})}, \tag{56}$$

原文第 34 页

样本来自旧推理策略 π_o。对该目标求导，会得到之前推导的 token 重加权策略梯度估计器，再加上 GRPO 的优势与序列归一化。

记 $A^{(i,j)}=[r(y^{(i,j)}|x^{(i)})-\mu_i]/\text{std}_i$ 为第 i 个提示词第 j 个回答的优势，$w_t^{(i,j)}=\pi\theta(y_t|x,y<t)/\pi_o(y_t|x,y<t)$ 为第 t 个重要性权重，则裁剪后的目标为：

$$J_\theta^{\text{GRPO-off-policy-clip}} = \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{1}{\text{len}\left(y^{(i,j)}\right)}\sum_{t=1}^{\text{len}\left(y^{(i,j)}\right)}\min\left(A^{(i,j)}w_t^{(i,j)}, A^{(i,j)}\ \text{clip}\left(w_t^{(i,j)}, [1-\varepsilon, 1+\varepsilon]\right)\right), \tag{57}$$

原文第 34 页

$\text{clip}(w,[1-\varepsilon,1+\varepsilon])=\min(\max(w,1-\varepsilon),1+\varepsilon)$，即将权重限制在区间 $[1-\varepsilon,1+\varepsilon]$。更直观的等价写法如下 [J. Achiam，2018]：

$$J_\theta^{\text{GRPO-off-policy-clip}} = \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{1}{\text{len}\left(y^{(i,j)}\right)}\sum_{t=1}^{\text{len}\left(y^{(i,j)}\right)}\begin{cases}\min\left(w_t^{(i,j)}, 1+\varepsilon\right)A^{(i,j)} & \text{if}\quad A^{(i,j)}\geq 0\\ \max\left(w_t^{(i,j)}, 1-\varepsilon\right)A^{(i,j)} & \text{if}\quad A^{(i,j)}<0\end{cases}. \tag{58}$$

原文第 35 页

对于非负优势，模型会提高相应动作的概率，直到概率比达到 1+ε；之后 min 选择常数分支，梯度变为零。对于负优势，模型会降低动作概率，直到概率比达到 1−ε，之后该动作的梯度也变为零。因此梯度估计器为：

$$\nabla_\theta J_\theta^{\text{GRPO-off-policy-clip}} = \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{1}{\text{len}\left(y^{(i,j)}\right)}\sum_{t=1}^{\text{len}\left(y^{(i,j)}\right)}\text{mask}_t^{(i,j)}w_t^{(i,j)}A^{(i,j)}\nabla_\theta\log\pi_\theta(y_t \mid x, y_{<t}) \tag{59}$$

原文第 35 页

其中掩码为：

$$\text{mask}_t^{(i,j)} = \begin{cases}\mathbb{1}\left\{w_t^{(i,j)}<1+\varepsilon\right\} & \text{if}\quad A^{(i,j)}\geq 0\\ \mathbb{1}\left\{w_t^{(i,j)}>1-\varepsilon\right\} & \text{if}\quad A^{(i,j)}<0\end{cases} \tag{60}$$

原文第 35 页

即正优势时屏蔽过大的重要性权重，负优势时屏蔽过小的权重。

现在可以实现 importance_reweighting_method="noclip" 和 "grpo"，分别对应不裁剪与裁剪的异策略目标。与同策略情形一样，损失函数应返回目标的负值，让梯度下降完成目标上的梯度上升。

题目（compute_policy_gradient_loss_off_policy）：采用 token 级重加权的异策略策略梯度（1 分）

交付内容：扩展 compute_policy_gradient_loss，支持 importance_reweighting_method="noclip" 和 "grpo"，使用 old_log_probs 与 cliprange 参数。old_log_probs 为生成 rollout 的模型对每个 token 给出的对数概率，需要在训练脚本中增加几行来计算；cliprange 为裁剪强度 ε。

实现 adapters.run_compute_policy_gradient_loss，运行 uv run pytest -k test_compute_policy_gradient_loss_off_policy，确保通过测试。

说明：PPO 将裁剪解释为防止当前策略过度偏离旧策略；由于它压制过大的重要性权重，也可以视作降低估计方差的方法。如果不太在意偏离旧策略，并希望更激进地更新，更简单直接的方差控制办法是在梯度估计器中直接对重要性权重设置上界：

$$\hat{g} \leftarrow \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{1}{\operatorname{len}(y^{(i,j)})}\sum_{t=1}^{\operatorname{len}(y^{(i,j)})}\min\left(w_t^{(i,j)}, 1+\varepsilon\right)A^{(i,j)}\nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t}), \tag{61}$$

原文第 36 页

与 PPO/GRPO 不同，即使动作的概率比超过 1+ε，这里仍会得到非零梯度。CISPO [MiniMax 等，2025] 提出了这种做法。它还按组内 token 数归一化，而不是按每条序列长度；这里为简化展示仍写成序列归一化形式。你可以选做 CISPO，并与其他裁剪方法比较。

6.3 GSPO

token 级重加权忽略了前缀和后缀的重加权，因此存在偏差。GSPO 论文 [C. Zheng 等，2025] 观察到 GRPO 不稳定，主张采用序列级重加权。为避免 L 个权重连乘带来的问题，他们将序列权重取 1/L 次幂，即用各 token 重要性权重的几何平均数代替乘积，得到：

$$J_\theta^{\text{GRPO-off-policy-gspo}} = \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\min\left(A^{(i,j)}s^{(i,j)}, A^{(i,j)}\ \operatorname{clip}\left(s^{(i,j)}, [1-\varepsilon, 1+\varepsilon]\right)\right), \tag{62}$$

原文第 36 页

序列级权重 s^(i,j) 在所有时间步之间共享：

$$s^{(i,j)} = \left(\prod_{t=1}^{\operatorname{len}(y^{(i,j)})}\frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_0(y_t \mid x, y_{<t})}\right)^{\frac{1}{\operatorname{len}(y^{(i,j)})}}. \tag{63}$$

原文第 36 页

若去掉 1/len(y^(i,j)) 次幂且不裁剪，这就是完整序列级重要性重加权的策略梯度损失。几何平均与裁剪以引入偏差为代价降低方差。求梯度时，几何平均的幂次还会自然引入序列长度归一化；为简洁，下面忽略裁剪：

$$\nabla_\theta J_\theta^{\text{GRPO-off-policy-gspo}} = \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G} A^{(i,j)} \nabla_\theta s^{(i,j)} \tag{64}$$

$$= \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G} A^{(i,j)} s^{(i,j)} \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t})). \tag{65}$$

原文第 36 页

因此，如果想将 GSPO 应用于采用常数归一化的 RL 算法（选做），需要调整序列级重要性权重的指数。

题目（think_about_importance_reweighting）：思考重要性重加权（2 分）

考虑三种异策略 RL 策略：(a) 不做重要性重加权；(b) PPO/GRPO 风格、带裁剪的 token 级重加权；(c) GSPO 风格、使用几何平均并裁剪的序列级重加权。它们分别处于偏差—方差权衡的什么位置？能否举出直觉上某种方法更适合的情形？

交付内容：几句分析。

接下来实现 GSPO 损失。请用数值稳定的方式计算几何平均。

题目（compute_policy_gradient_loss_off_policy_gspo）：采用序列级重加权的异策略策略梯度（1 分）

交付内容：扩展 compute_policy_gradient_loss，支持 importance_reweighting_method="gspo"，使用 old_log_probs 和 cliprange。前者是生成 rollout 的模型对每个 token 给出的对数概率，需在训练脚本中增加计算；后者是裁剪强度 ε。同样返回目标的负值，使最小化损失等价于梯度上升。

实现 adapters.run_compute_policy_gradient_loss，运行 uv run pytest -k test_compute_policy_gradient_loss_off_policy_gspo，确保通过测试。

6.4 实验

现在组件已齐备，可以让 grpo_train_step 支持异策略训练。

题目（grpo_train_step_off_policy）：异策略 GRPO 训练步骤（2.5 分）

交付内容：扩展 grpo_train_step，支持异策略参数 importance_reweighting_method、old_log_probs、cliprange。至此，函数应支持全部参数组合。

实现 adapters.run_grpo_train_step，运行 uv run pytest -k test_grpo_train_step_off_policy，确保通过测试。

异策略训练是否真的更快？会牺牲多少稳定性？哪种重加权/裁剪最好？下面的实验将探索这些问题。除改为每个推理批次做 32 次训练更新外，其余超参数与之前的 RL 运行保持一致：

rollout_batch_size = 256；train_batch_size = 8；gradient_accumulation_steps = 1。

默认裁剪范围可采用原论文的值：offpolicy_clip 使用 cliprange=0.2；offpolicy_gspo 使用 cliprange=3e-4。也可以自行调节。记得在训练脚本中计算 old_log_probs，并传给重加权损失。

题目（grpo_experiments_off_policy）：比较不同异策略算法的表现（8 B200 小时，10 分）

保持与标准 GRPO 相同的超参数，学习率采用你选定的值。每个变体使用 4 个随机种子、零样本 r1_zero 提示词。也可以选用上一节中你最喜欢的 RL 变体作为基础。

• offpolicy_naive：将训练批量从与推理批量相同的 256 减至其 1/32，即 8；梯度累积步数也按相同比例减少，以继续充分利用 GPU。保持 importance_reweighting_method="none"。

• offpolicy_noclip：在 offpolicy_naive 基础上设为 "noclip"。

• offpolicy_clip：在 offpolicy_naive 基础上设为 "grpo"。

• offpolicy_gspo：在 offpolicy_naive 基础上设为 "gspo"。

除此前指标外，还要记录裁剪比例（clip fraction）。与完全同策略 GRPO 相比，这些方法表现如何？异策略训练对稳定性和运行间方差有何影响？两种裁剪方法的裁剪比例有何差别，哪种更稳定？哪些方法可能受益于进一步调参？结合运行之间的方差，你对结论有多大把握？

交付内容：分析说明，并用所引用指标的图表支持结论。

7 尝试你自己的策略梯度估计器

题目（try_your_own）：尝试自己的策略梯度估计器（10 分）

你已见过文献中的多种策略梯度估计器及其部分理论，现在提出你自己的方法！例如，可以采用不同的优势估计器，产生不同的提示词权重；采用不同的重要性重加权策略；或在优势估计器中使用不同的奖励基线。

提出自己的策略梯度估计器，在相同的 OLMo-2-0425-1B / GSM8K RL 设置下运行，并与之前的方法比较。使用多个随机种子，固定问题设置以保证可比性。相对于已有方法，只改变一项：至少应存在本作业介绍过的一种方法，与你的方法仅有一个不同点。

你提出的改动有何理论依据或直觉？它是否胜过基线？

交付内容：分析说明，并用相关指标图表支持；如数学推导有助于论证，也可一并提供。

参考文献

参考文献题名保留原文，以便检索。

[1] J. Li 等，DataComp-LM: In search of the next generation of training sets for language models. https://arxiv.org/abs/2406.11794

[2] T. OLMo 等，2 OLMo 2 Furious. https://arxiv.org/abs/2501.00656

[3] K. Cobbe 等，Training Verifiers to Solve Math Word Problems. https://arxiv.org/abs/2110.14168

[4] DeepSeek-AI 等，DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. https://arxiv.org/abs/2501.12948

[5] W. Kwon 等，Efficient Memory Management for Large Language Model Serving with PagedAttention. 2023.

[6] Z. Liu 等，Understanding R1-Zero-Like Training: A Critical Perspective. https://arxiv.org/abs/2503.20783

[7] OpenAI 等，OpenAI o1 System Card. https://arxiv.org/abs/2412.16720

[8] Z. Shao 等，DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. https://arxiv.org/abs/2402.03300

[9] J. Achiam，Spinning Up in Deep Reinforcement Learning. 2018.

[10] N. Lambert，Reinforcement Learning from Human Feedback. https://rlhfbook.com/

[11] R. J. Williams，Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning, 8(3–4):229–256, 1992. doi:10.1007/BF00992696.

[12] F. Tajwar 等，Maximum Likelihood Reinforcement Learning. https://arxiv.org/abs/2602.02710

[13] J. Schulman、F. Wolski、P. Dhariwal、A. Radford、O. Klimov，Proximal Policy Optimization Algorithms. https://arxiv.org/abs/1707.06347

[14] T. Degris、M. White、R. S. Sutton，Off-Policy Actor-Critic. https://arxiv.org/abs/1205.4839

[15] J. Achiam，Simplified PPO-Clip Objective. https://drive.google.com/file/d/1PDzn9RPvaXjJFZkGeapMHbHGiWWW20Ey/view

[16] MiniMax 等，MiniMax-M1: Scaling Test-Time Compute Efficiently with Lightning Attention. https://arxiv.org/abs/2506.13585

[17] C. Zheng 等，Group Sequence Policy Optimization. https://arxiv.org/abs/2507.18071